

# CSE312: Database Management Systems Lab

## Lab 02: Introduction to PostgreSQL and SQL DDL Statements

### 1. Objective

The objectives of this experiment are:

- To become familiar with PostgreSQL and basic database navigation commands.
- To understand relational database concepts such as tables, columns, rows, keys, and constraints.
- To create tables using PostgreSQL Data Definition Language (DDL) statements.
- To modify table structures using essential `ALTER TABLE` commands.
- To use `DROP` and `TRUNCATE` commands safely.
- To insert sample records and view table data for verification.

### 2. Introduction to PostgreSQL

**PostgreSQL:** PostgreSQL is an open-source object-relational database management system. It is widely used for storing, organizing, and managing structured data.

PostgreSQL supports standard SQL along with many advanced database features. In this lab, students will use PostgreSQL to create a small relational database and practice basic database commands.

#### Required software:

- PostgreSQL Server
- pgAdmin or psql command-line tool

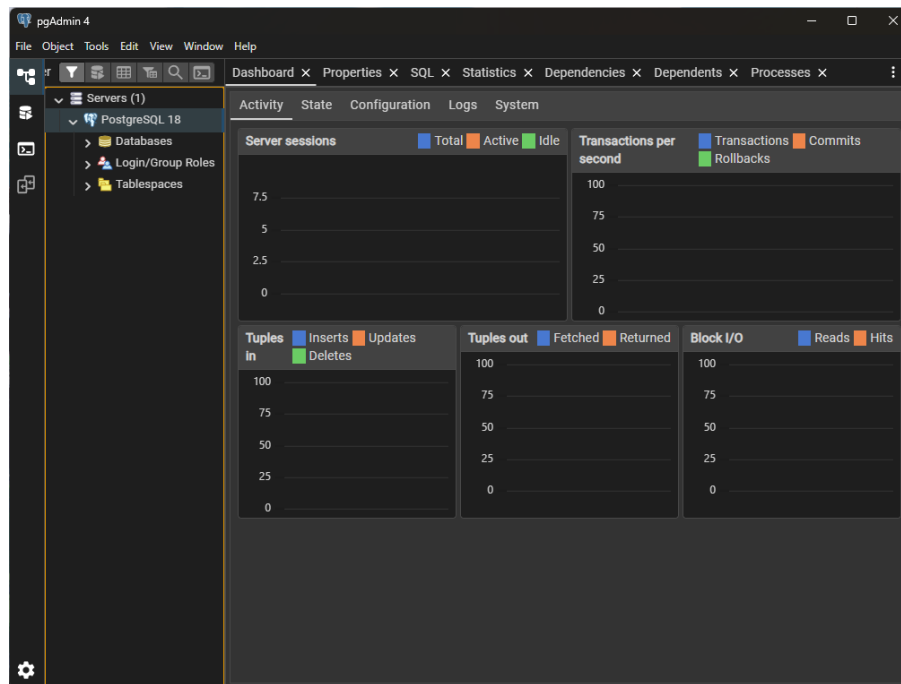


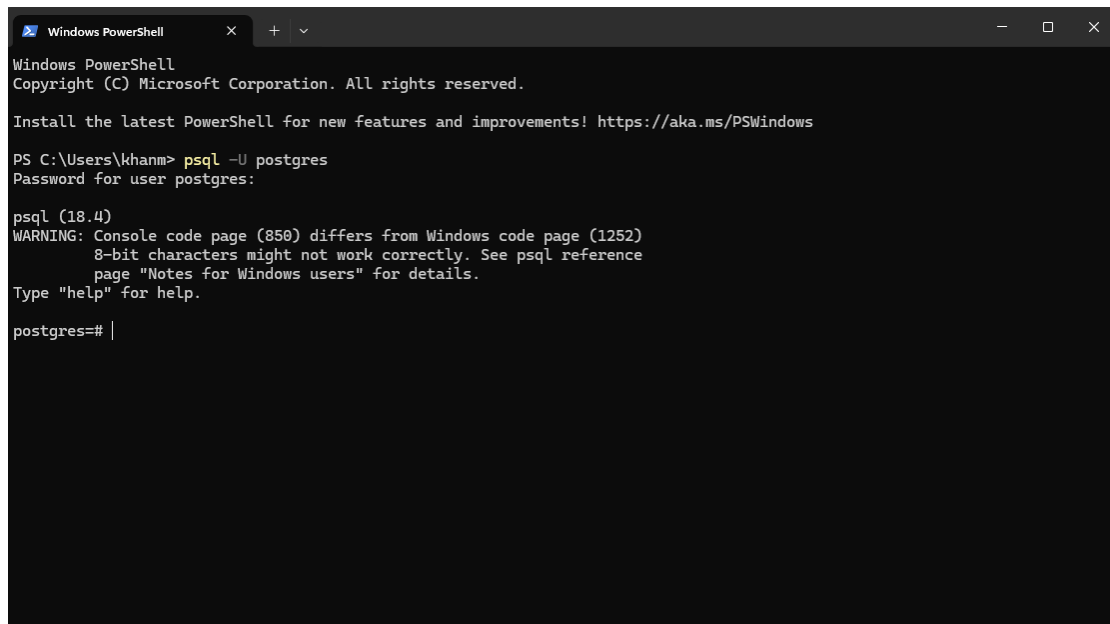
Figure 1: pgAdmin dashboard for managing PostgreSQL databases

## 2.1 Connecting to PostgreSQL

The following command opens the `psql` terminal and connects to PostgreSQL using the `postgres` user. You have to make sure that the PostgreSQL server is running and the `postgres` user has the correct password set up.

*Note:* If your terminal says it cannot find the `psql` command, you may need to add PostgreSQL's `bin` directory to your system's `PATH` environment variable.

```
psql -U postgres
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\khanm> psql -U postgres
Password for user postgres:

psql (18.4)
WARNING: Console code page (850) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

postgres=# |
```

Figure 2: Successful login to PostgreSQL using the psql terminal

## 2.2 Basic psql Navigation Commands

Before using navigation commands, it is important to understand the database objects being navigated.

**Database:** A database is an organized collection of related data. In PostgreSQL, a database contains schemas, tables, and other objects used to store and manage information for a particular application or purpose.

**Schema:** A schema is a logical container inside a database. It groups related database objects such as tables, views, and functions, and helps organize large databases.

**Table:** A table is a database object that stores data in rows and columns. Each table usually represents one entity, such as students, departments, courses, or employees.

**Example:** In a `university_lab` database, an `academics` schema may contain a `student` table.

After connecting to PostgreSQL, use the following commands to navigate databases, schemas, and tables.

| Command                       | Purpose                                          |
|-------------------------------|--------------------------------------------------|
| <code>\l</code>               | Lists all databases.                             |
| <code>\c database_name</code> | Connects to a selected database.                 |
| <code>\dt</code>              | Lists all tables in the current database schema. |
| <code>\d table_name</code>    | Shows the structure of a selected table.         |
| <code>\dn</code>              | Lists available schemas.                         |
| <code>\q</code>               | Exits psql.                                      |

Table 1: Basic psql navigation commands

```

psql (18.4)
WARNING: Console code page (850) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

postgres=# \l
          List of databases
  Name | Owner | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 postgres | postgres | UTF8 | libc | English_United States.1252 | English_United States.1252 |  |  | =c/postgres
 template0 | postgres | UTF8 | libc | English_United States.1252 | English_United States.1252 |  |  | =c/postgres
 template1 | postgres | UTF8 | libc | English_United States.1252 | English_United States.1252 |  |  | =c/postgres
(3 rows)

postgres=# \c postgres
You are now connected to database "postgres" as user "postgres".
postgres=# \dt
Did not find any tables.
postgres=# \dn
          List of schemas
  Name | Owner
-----+-----
 public | pg_database_owner
(1 row)

postgres=# \q
PS C:\Users\khanm>

```

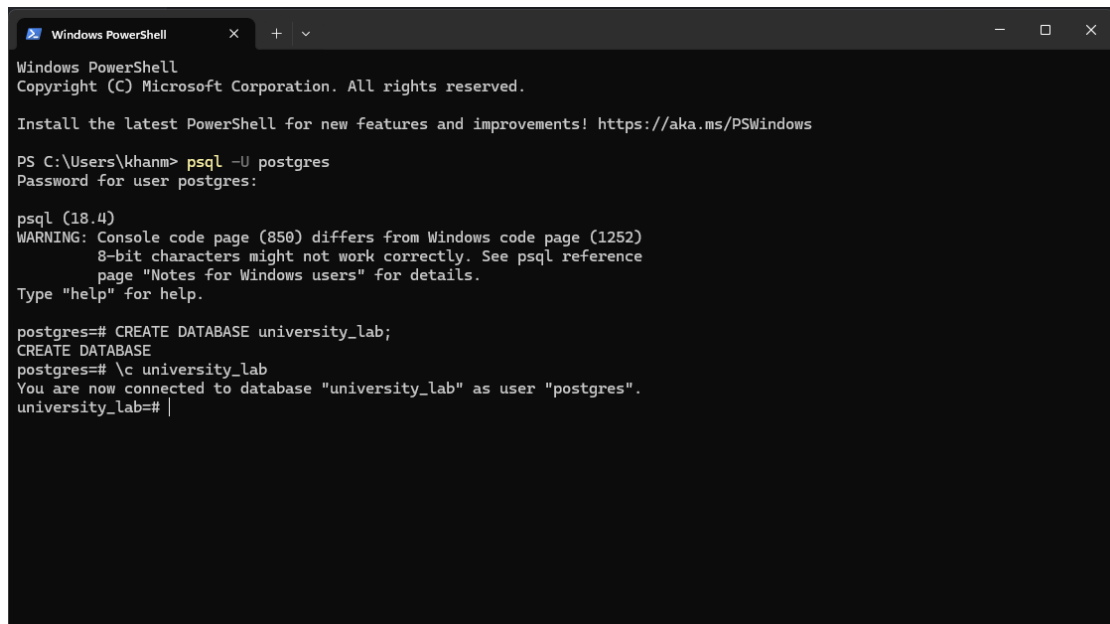
Figure 3: Output of basic psql navigation commands

## 2.3 Creating and Selecting a Database

Create a database for this lab and connect to it.

```
CREATE DATABASE university_lab;
```

```
\c university_lab
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\khanm> psql -U postgres
Password for user postgres:

psql (18.4)
WARNING: Console code page (850) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

postgres=# CREATE DATABASE university_lab;
CREATE DATABASE
postgres=# \c university_lab
You are now connected to database "university_lab" as user "postgres".
university_lab=#
```

Figure 4: Creating and connecting to the `university_lab` database

## 3. Relational Database Concepts

A relational database stores data in tables. Each table represents an entity, and relationships between tables are created using keys.

### 3.1 Tables, Rows, and Columns

**Table:** A table is a collection of related data arranged in rows and columns. For example, a `student` table stores student information.

**Column:** A column represents a specific attribute of a table, such as `student_id`, `name`, or `email`.

**Row:** A row represents one record in a table. For example, one row in the `student` table represents one student.

```

university_lab=# INSERT INTO department (dept_name, location)
university_lab=# VALUES ('Computer Science and Engineering', 'Academic Building 1');
INSERT 0 1
university_lab=# INSERT INTO student
university_lab=# (first_name, last_name, email, date_of_birth, admission_year, dept_id)
university_lab=# VALUES
university_lab=# ('Rahim', 'Ahmed', 'rahim.ahmed@example.com', '2002-05-14', 2023, 1);
INSERT 0 1
university_lab=# INSERT INTO course (course_code, course_title, credit, dept_id)
university_lab=# VALUES ('CSE312', 'Database Management Systems Lab', 1.5, 1);
INSERT 0 1
university_lab=# INSERT INTO enrollment (student_id, course_id, grade)
university_lab=# VALUES (1, 1, 'A');
INSERT 0 1
university_lab=# SELECT * FROM department;
 dept_id | dept_name | location
-----
1 | Computer Science and Engineering | Academic Building 1
(1 row)

university_lab=# SELECT * FROM student;
 student_id | first_name | last_name | email | date_of_birth | admission_year | is_active | dept_id | mobile_number
-----
1 | Rahim | Ahmed | rahim.ahmed@example.com | 2002-05-14 | 2023 | t | 1 |
(1 row)

university_lab=# SELECT * FROM course;
 course_id | course_code | course_title | credit | dept_id
-----
1 | CSE312 | Database Management Systems Lab | 1.5 | 1
(1 row)

university_lab=# SELECT * FROM enrollment;
 student_id | course_id | enrollment_date | grade
-----
1 | 1 | 2026-06-03 | A
(1 row)

university_lab=#

```

Figure 5: Example table output showing rows and columns

## 3.2 Common PostgreSQL Data Types

Each column must have a data type. The data type defines what kind of value can be stored in the column.

| Data Type            | Description                                                     |
|----------------------|-----------------------------------------------------------------|
| <b>INTEGER</b>       | Stores whole numbers.                                           |
| <b>SERIAL</b>        | Auto-incrementing integer, commonly used for primary keys.      |
| <b>VARCHAR(n)</b>    | Stores variable-length character strings with a maximum length. |
| <b>TEXT</b>          | Stores variable-length text without a fixed maximum length.     |
| <b>NUMERIC(p, s)</b> | Stores exact decimal values with precision and scale.           |
| <b>DATE</b>          | Stores calendar dates.                                          |
| <b>TIMESTAMP</b>     | Stores date and time values.                                    |
| <b>BOOLEAN</b>       | Stores true or false values.                                    |

Table 2: Frequently used PostgreSQL data types

### 3.3 Keys and Constraints

**Key:** A key is a column or group of columns used to identify records or create relationships between tables.

**Constraint:** A constraint is a rule applied to a table column or a group of columns to maintain data accuracy and integrity.

Common keys and constraints include:

- **PRIMARY KEY:** uniquely identifies each row in a table.
- **FOREIGN KEY:** links a column to the primary key of another table.
- **NOT NULL:** prevents a column from storing null values.
- **UNIQUE:** ensures all values in a column are different.
- **CHECK:** ensures values satisfy a specified condition.
- **DEFAULT:** assigns a value automatically when no value is provided.

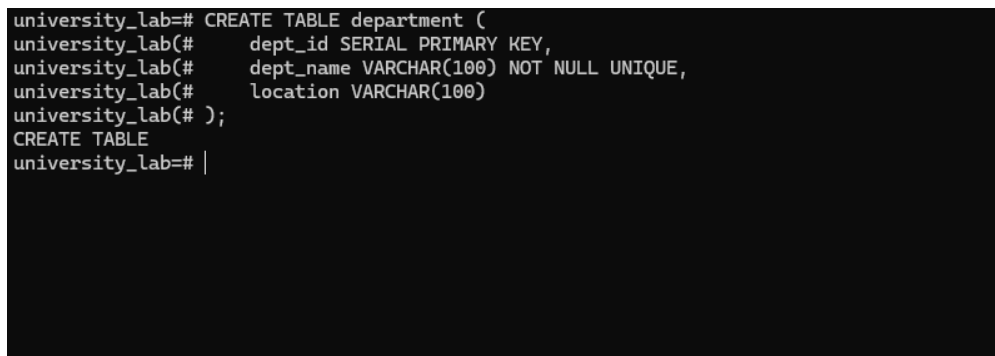
## 4. Creating Tables with DDL Statements

**DDL:** Data Definition Language is the subset of SQL used to define and manage database structures such as tables, schemas, and constraints.

The `CREATE TABLE` statement creates a new table. The statement includes table name, column names, data types, and constraints.

### 4.1 Creating a Simple Table

```
CREATE TABLE department (  
    dept_id SERIAL PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL UNIQUE,  
    location VARCHAR(100)  
);
```



```
university_lab=# CREATE TABLE department (  
university_lab(#      dept_id SERIAL PRIMARY KEY,  
university_lab(#      dept_name VARCHAR(100) NOT NULL UNIQUE,  
university_lab(#      location VARCHAR(100)  
university_lab(# );  
CREATE TABLE  
university_lab=# |
```

Figure 6: Creating the `department` table

### 4.2 Creating a Table with Different Data Types and Constraints

```
CREATE TABLE student (  
    student_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    date_of_birth DATE,  
    admission_year INTEGER CHECK (admission_year >= 2000),  
    is_active BOOLEAN DEFAULT TRUE,  
    dept_id INTEGER REFERENCES department(dept_id)  
);
```



```

university_lab=# CREATE TABLE student (
university_lab(#     student_id SERIAL PRIMARY KEY,
university_lab(#     first_name VARCHAR(50) NOT NULL,
university_lab(#     last_name VARCHAR(50) NOT NULL,
university_lab(#     email VARCHAR(100) UNIQUE,
university_lab(#     date_of_birth DATE,
university_lab(#     admission_year INTEGER CHECK (admission_year >= 2000),
university_lab(#     is_active BOOLEAN DEFAULT TRUE,
university_lab(#     dept_id INTEGER REFERENCES department(dept_id)
university_lab(# );
CREATE TABLE
university_lab=# |

```

Figure 7: Creating the student table

### 4.3 Creating a Table with a Foreign Key

```

CREATE TABLE course (
    course_id SERIAL PRIMARY KEY,
    course_code VARCHAR(20) NOT NULL UNIQUE,
    course_title VARCHAR(150) NOT NULL,
    credit NUMERIC(3, 1) CHECK (credit > 0),
    dept_id INTEGER NOT NULL REFERENCES department(dept_id)
);

```

### 4.4 Creating a Table with Composite Primary Key

A composite primary key is formed using two or more columns. It is useful for junction tables that represent many-to-many relationships.

```

CREATE TABLE enrollment (
    student_id INTEGER REFERENCES student(student_id),
    course_id INTEGER REFERENCES course(course_id),
    enrollment_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),
    PRIMARY KEY (student_id, course_id)
);

```

```

university_lab=# CREATE TABLE course (
university_lab(#   course_id SERIAL PRIMARY KEY,
university_lab(#   course_code VARCHAR(20) NOT NULL UNIQUE,
university_lab(#   course_title VARCHAR(150) NOT NULL,
university_lab(#   credit NUMERIC(3, 1) CHECK (credit > 0),
university_lab(#   dept_id INTEGER NOT NULL REFERENCES department(dept_id)
university_lab(# );
university_lab=# CREATE TABLE
university_lab=# CREATE TABLE enrollment (
university_lab(#   student_id INTEGER REFERENCES student(student_id),
university_lab(#   course_id INTEGER REFERENCES course(course_id),
university_lab(#   enrollment_date DATE DEFAULT CURRENT_DATE,
university_lab(#   grade VARCHAR(2),
university_lab(#   PRIMARY KEY (student_id, course_id)
university_lab(# );
university_lab=# CREATE TABLE
university_lab=#

```

Figure 8: Creating the course and enrollment tables

## 4.5 Viewing Table Structure

After creating tables, inspect their structure using psql.

```

\dt
\d department
\d student
\d course
\d enrollment

```

```

Windows PowerShell
university_lab=#
university_lab=# \dt
List of tables
Schema | Name | Type | Owner
-----|-----|-----|-----
public | course | table | postgres
public | department | table | postgres
public | enrollment | table | postgres
public | student | table | postgres
(4 rows)

university_lab=# \d department
Table "public.department"
Column | Type | Collation | Nullable | Default
-----|-----|-----|-----|-----
dept_id | integer | | not null | nextval('department_dept_id_seq'::regclass)
dept_name | character varying(100) | | not null | 
location | character varying(100) | | not null | 
Indexes:
    "department_pkey" PRIMARY KEY, btree (dept_id)
    "department_dept_name_key" UNIQUE CONSTRAINT, btree (dept_name)
Referenced by:
    TABLE "course" CONSTRAINT "course_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES department(dept_id)
    TABLE "student" CONSTRAINT "student_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES department(dept_id)

university_lab=# \d student
Table "public.student"
Column | Type | Collation | Nullable | Default
-----|-----|-----|-----|-----
student_id | integer | | not null | nextval('student_student_id_seq'::regclass)
first_name | character varying(50) | | not null | 
last_name | character varying(50) | | not null | 
email | character varying(100) | | not null | 
date_of_birth | date | | | 
admission_year | integer | | | 
is_active | boolean | | true | 
dept_id | integer | | | 
Indexes:
    "student_pkey" PRIMARY KEY, btree (student_id)
    "student_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "student_admission_year_check" CHECK (admission_year >= 2000)
Foreign-key constraints:
    "student_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES department(dept_id)
Referenced by:
    TABLE "enrollment" CONSTRAINT "enrollment_student_id_fkey" FOREIGN KEY (student_id) REFERENCES student(student_id)

```

Figure 9: Viewing table lists and table structures in psql

## 5. Modifying Tables with ALTER

The ALTER TABLE statement modifies the structure of an existing table.

### 5.1 Adding a Column

```

ALTER TABLE student
ADD COLUMN phone VARCHAR(20);

```

### 5.2 Deleting a Column

```

ALTER TABLE student
DROP COLUMN phone;

```

### 5.3 Changing a Column Data Type

```

ALTER TABLE student

```

```
ADD COLUMN contact_number VARCHAR(20);
```

```
ALTER TABLE student
```

```
ALTER COLUMN contact_number TYPE VARCHAR(30);
```

## 5.4 Renaming a Column

```
ALTER TABLE student
```

```
RENAME COLUMN contact_number TO mobile_number;
```

## 5.5 Adding a Constraint

```
ALTER TABLE enrollment
```

```
ADD CONSTRAINT grade_check
```

```
CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F'));
```

```
university_lab=# ALTER TABLE student
university_lab=# DROP COLUMN phone;
ALTER TABLE
university_lab=# ALTER TABLE student
university_lab=# ADD COLUMN contact_number VARCHAR(20);
ALTER TABLE
university_lab=# ALTER TABLE student
university_lab=# ALTER COLUMN contact_number TYPE VARCHAR(30);
ALTER TABLE
university_lab=# ALTER TABLE student
university_lab=# RENAME COLUMN contact_number TO mobile_number;
ALTER TABLE
university_lab=# ALTER TABLE enrollment
university_lab=# ADD CONSTRAINT grade_check
university_lab=# CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F'));
ALTER TABLE
```

Figure 10: Executing ALTER TABLE commands

```
university_lab=# \d student
```

| Column         | Type                   | Collation | Nullable | Default                                     |
|----------------|------------------------|-----------|----------|---------------------------------------------|
| student_id     | integer                |           | not null | nextval('student_student_id_seq'::regclass) |
| first_name     | character varying(50)  |           | not null |                                             |
| last_name      | character varying(50)  |           | not null |                                             |
| email          | character varying(100) |           |          |                                             |
| date_of_birth  | date                   |           |          |                                             |
| admission_year | integer                |           |          |                                             |
| is_active      | boolean                |           |          | true                                        |
| dept_id        | integer                |           |          |                                             |
| mobile_number  | character varying(30)  |           |          |                                             |

```
Indexes:
    "student_pkey" PRIMARY KEY, btree (student_id)
    "student_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "student_admission_year_check" CHECK (admission_year >= 2000)
Foreign-key constraints:
    "student_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES department(dept_id)
Referenced by:
    TABLE "enrollment" CONSTRAINT "enrollment_student_id_fkey" FOREIGN KEY (student_id) REFERENCES student(student_id)
```

Figure 11: Updated student table structure after ALTER commands

```

university_lab=# \d enrollment
                                Table "public.enrollment"
    Column      |      Type      | Collation | Nullable | Default
-----|-----|-----|-----|-----
 student_id     | integer         |           | not null |
 course_id      | integer         |           | not null |
 enrollment_date | date            |           |          | CURRENT_DATE
 grade          | character varying(2) |           |          |
Indexes:
    "enrollment_pkey" PRIMARY KEY, btree (student_id, course_id)
Check constraints:
    "grade_check" CHECK (grade::text = ANY (ARRAY['A+ '::character varying, 'A'::character varying, 'B+ '::character varying, 'B'::character varying, 'C+ '::character varying, 'C'::character varying, 'D'::character varying, 'F'::character varying]::text[]))
Foreign-key constraints:
    "enrollment_course_id_fkey" FOREIGN KEY (course_id) REFERENCES course(course_id)
    "enrollment_student_id_fkey" FOREIGN KEY (student_id) REFERENCES student(student_id)

university_lab=#

```

Figure 12: Updated enrollment table structure after adding a constraint

## 5.6 More ALTER Commands for Further Exploration

The following commands are useful in real database work. Students should explore them after practicing the essential ALTER TABLE commands above and figure out what they do.

```
ALTER TABLE student RENAME TO university_student;
```

```
ALTER TABLE university_student
ALTER COLUMN email SET NOT NULL;
```

```
ALTER TABLE university_student
ALTER COLUMN is_active SET DEFAULT TRUE;
```

```
ALTER TABLE university_student
DROP CONSTRAINT IF EXISTS university_student_email_key;
```

## 6. DROP and TRUNCATE Commands

### 6.1 DROP TABLE

The DROP TABLE command removes a table permanently. Use it carefully because the table definition and its data are deleted.

```
DROP TABLE IF EXISTS temporary_table;
```

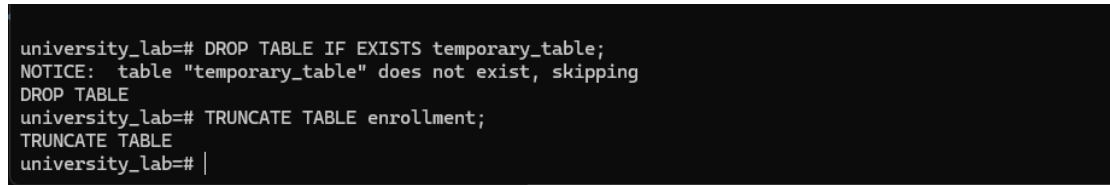
### 6.2 TRUNCATE TABLE

The TRUNCATE command removes all rows from a table but keeps the table structure.

```
TRUNCATE TABLE enrollment;
```

If another table references the table, `CASCADE` may be needed. Use it carefully.

```
TRUNCATE TABLE enrollment CASCADE;
```

A terminal window with a dark background and light-colored text. The text shows a series of SQL commands and their outputs. The commands are: 'DROP TABLE IF EXISTS temporary\_table;', 'TRUNCATE TABLE enrollment;', and 'TRUNCATE TABLE'. The outputs include a notice that the table 'temporary\_table' does not exist, and a prompt 'university\_lab=#' followed by a vertical bar '|'.

```
university_lab=# DROP TABLE IF EXISTS temporary_table;
NOTICE: table "temporary_table" does not exist, skipping
DROP TABLE
university_lab=# TRUNCATE TABLE enrollment;
TRUNCATE TABLE
university_lab=# |
```

Figure 13: Executing DROP and TRUNCATE commands

## 7. Inserting and Viewing Table Data

Although `INSERT` and `SELECT` are not DDL commands, they are useful for checking whether the created table structures work correctly.

### 7.1 Inserting Records

```
INSERT INTO department (dept_name, location)
VALUES ('Computer Science and Engineering', 'Academic Building 1');

INSERT INTO student
    (first_name, last_name, email, date_of_birth, admission_year, dept_id)
VALUES
    ('Rahim', 'Ahmed', 'rahim.ahmed@example.com', '2002-05-14', 2023, 1);

INSERT INTO course (course_code, course_title, credit, dept_id)
VALUES ('CSE312', 'Database Management Systems Lab', 1.5, 1);

INSERT INTO enrollment (student_id, course_id, grade)
VALUES (1, 1, 'A');
```

### 7.2 Viewing Records

```
SELECT * FROM department;
SELECT * FROM student;
SELECT * FROM course;
SELECT * FROM enrollment;
```

```

university_lab=# INSERT INTO department (dept_name, location)
university_lab=# VALUES ('Computer Science and Engineering', 'Academic Building 1');
INSERT 0 1
university_lab=# INSERT INTO student
university_lab=# (first_name, last_name, email, date_of_birth, admission_year, dept_id)
university_lab=# VALUES
university_lab=# ('Rahim', 'Ahmed', 'rahim.ahmed@example.com', '2002-05-14', 2023, 1);
INSERT 0 1
university_lab=# INSERT INTO course (course_code, course_title, credit, dept_id)
university_lab=# VALUES ('CSE312', 'Database Management Systems Lab', 1.5, 1);
INSERT 0 1
university_lab=# INSERT INTO enrollment (student_id, course_id, grade)
university_lab=# VALUES (1, 1, 'A');
INSERT 0 1
university_lab=# SELECT * FROM department;
 dept_id | dept_name | location
-----
1 | Computer Science and Engineering | Academic Building 1
(1 row)

university_lab=# SELECT * FROM student;
 student_id | first_name | last_name | email | date_of_birth | admission_year | is_active | dept_id | mobile_number
-----
1 | Rahim | Ahmed | rahim.ahmed@example.com | 2002-05-14 | 2023 | t | 1 |
(1 row)

university_lab=# SELECT * FROM course;
 course_id | course_code | course_title | credit | dept_id
-----
1 | CSE312 | Database Management Systems Lab | 1.5 | 1
(1 row)

university_lab=# SELECT * FROM enrollment;
 student_id | course_id | enrollment_date | grade
-----
1 | 1 | 2026-06-03 | A
(1 row)

university_lab=#

```

Figure 14: Inserted records displayed using SELECT

## 8. Observations

- PostgreSQL was accessed using pgAdmin or the psql command-line tool.
- Basic psql navigation commands were used to list databases, connect to a database, and inspect tables.
- Relational database concepts such as tables, rows, columns, data types, keys, and constraints were practiced.
- Tables were created using `CREATE TABLE` with suitable data types and constraints.
- Table structures were modified using essential `ALTER TABLE` commands.
- `DROP` and `TRUNCATE` commands were introduced for removing database objects or table records.
- Sample records were inserted and viewed to verify the created schema.

## 9. Lab Task

### Task A: Basic PostgreSQL Navigation

Complete the following tasks and attach screenshots:

1. Connect to PostgreSQL using pgAdmin or psql.
2. List all available databases.
3. Create a database named `library_lab`.
4. Connect to `library_lab`.
5. List all tables in the database.

## Task B: Library Management Tables

Create the following tables using PostgreSQL DDL statements:

1. `publisher`: stores publisher information.
2. `book`: stores book information and references `publisher`.
3. `member`: stores library member information.
4. `borrow`: stores borrowing records and references `book` and `member`.

Requirements:

- Use suitable data types for all columns.
- Define primary keys for all tables.
- Define foreign keys where necessary.
- Add at least one `UNIQUE`, one `NOT NULL`, one `CHECK`, and one `DEFAULT` constraint.
- Show each table structure using psql commands.

## Task C: ALTER, INSERT, and SELECT Practice

Using the tables created in Task B:

1. Add a new column to any table.
2. Rename one column.
3. Change the data type of one suitable column.
4. Delete one unnecessary column.
5. Insert at least two records into each table.
6. Display all records from each table using `SELECT`.



## Task D: Hospital Management Database

Write PostgreSQL statements for a hospital management database containing the following tables: `department`, `doctor`, `patient`, and `appointment`.

Requirements:

- Each doctor must belong to a department.
- Each appointment must reference one doctor and one patient.
- Appointment dates should have a default value of the current date.
- Consultation fees must be greater than or equal to zero.
- Use suitable primary key, foreign key, unique, not null, default, and check constraints.
- Insert sample records and display the final tables.

## 10. Conclusion

This experiment introduced PostgreSQL from basic navigation to relational table creation. Students learned how to connect to PostgreSQL, inspect databases and tables, define relational structures with data types, keys, and constraints, modify tables using `ALTER TABLE`, remove data or objects using `TRUNCATE` and `DROP`, and verify table structures by inserting and viewing sample records.